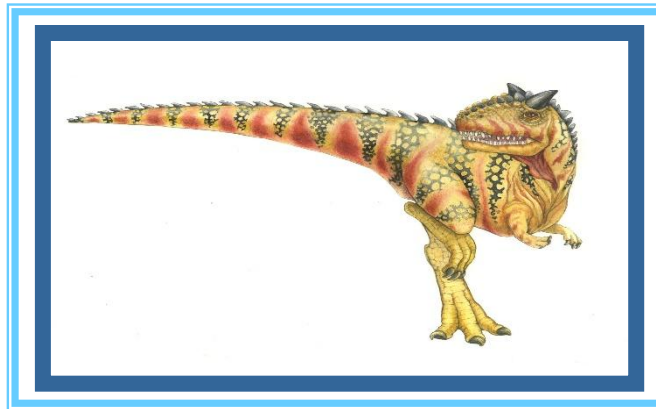


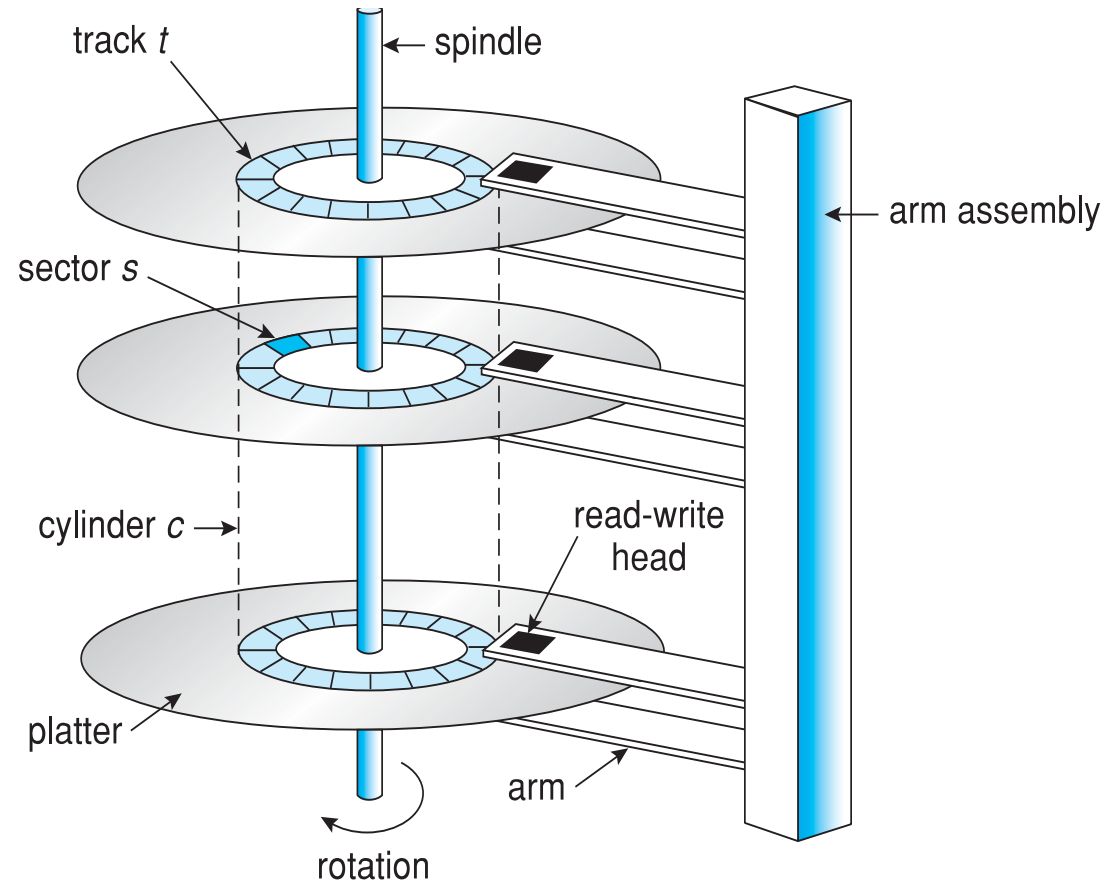
Chapter 11: Mass-Storage Systems



Overview of Mass Storage Structure

- Bulk of secondary storage for modern computers is **hard disk drives (HDDs)** and **nonvolatile memory (NVM)** devices
- **HDDs** spin platters of magnetically-coated material under moving read-write heads
 - Drives rotate at 60 to 250 times per second
 - **Transfer rate** is rate at which data flow between drive and computer
 - **Positioning time (random-access time)** is time to move disk arm to desired cylinder (**seek time**) and time for desired sector to rotate under the disk head (**rotational latency**)
 - **Head crash** results from disk head making contact with the disk surface -- That's bad
- Disks can be removable

Moving-head Disk Mechanism



Hard Disk Drives

- Platters range from .85” to 14” (historically)
 - Commonly 3.5”, 2.5”, and 1.8”
- Range from 30GB to 3TB per drive
- Performance
 - Transfer Rate – theoretical – 6 Gb/sec
 - Effective Transfer Rate – real – 1Gb/sec
 - Seek time from 3ms to 12ms – 9ms common for desktop drives
 - Average seek time measured or calculated based on 1/3 of tracks
 - Latency based on spindle speed
 - $1 / (\text{RPM} / 60) = 60 / \text{RPM}$
 - Average latency = $\frac{1}{2}$ latency



The First Commercial Disk Drive



1956
IBM RAMDAC computer
included the IBM Model 350
disk storage system

Disk Attachment

- Host-attached storage accessed through I/O ports talking to I/O busses
- Several busses available, including advanced technology attachment (ATA), serial ATA (SATA), eSATA, serial attached SCSI (SAS), universal serial bus (USB), and fibre channel (FC).
- Most common is SATA
- Because NVM much faster than HDD, new fast interface for NVM called NVM express (NVMe), connecting directly to PCI bus
- Data transfers on a bus carried out by special electronic processors called controllers (or host-bus adapters, HBAs)
 - Host controller on the computer end of the bus, device controller on device end
 - Computer places command on host controller, using memory-mapped I/O ports
 - Host controller sends messages to device controller
 - Data transferred via DMA between device and computer DRAM

HDD Scheduling

- The operating system is responsible for using hardware efficiently
 - For the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time $\approx$ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer
- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)

Disk I/O request

- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists

Queue Management

- In the past, operating system responsible for queue management, disk drive head scheduling
 - Now, built into the storage devices, controllers
 - Just provide Logical Block Addressing (LBA), handle sorting of requests
 - Some of the algorithms they use described next

Disk Scheduling Algorithms

- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

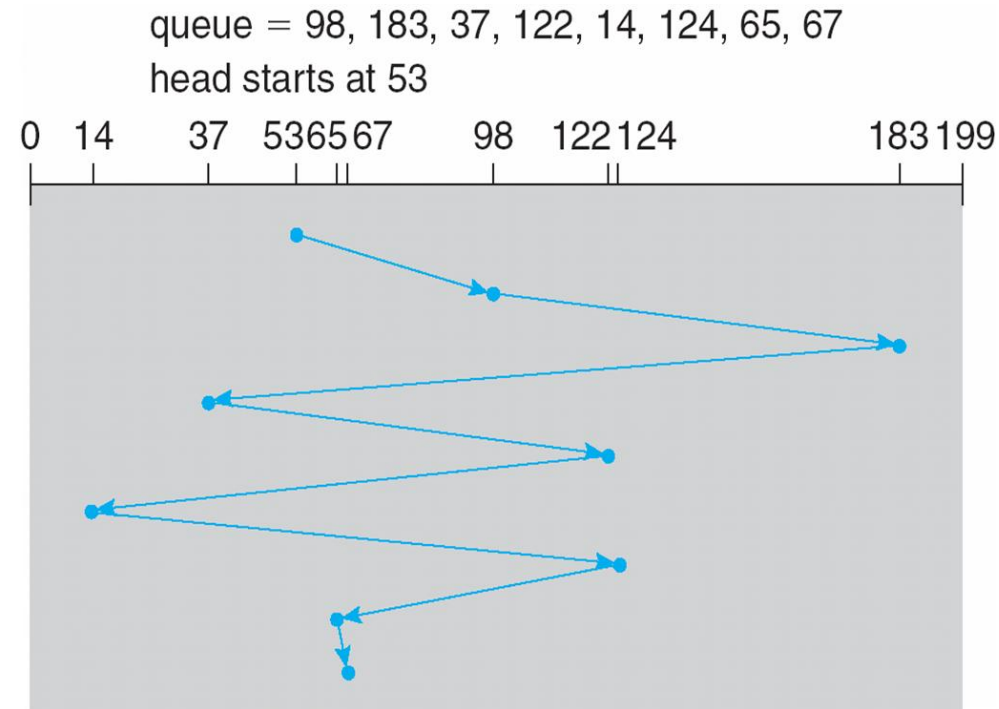
98, 183, 37, 122, 14, 124, 65, 67

Head at track 53

- Compare to algorithms to move an elevator in a building

First-come First-Serve (FCFS)

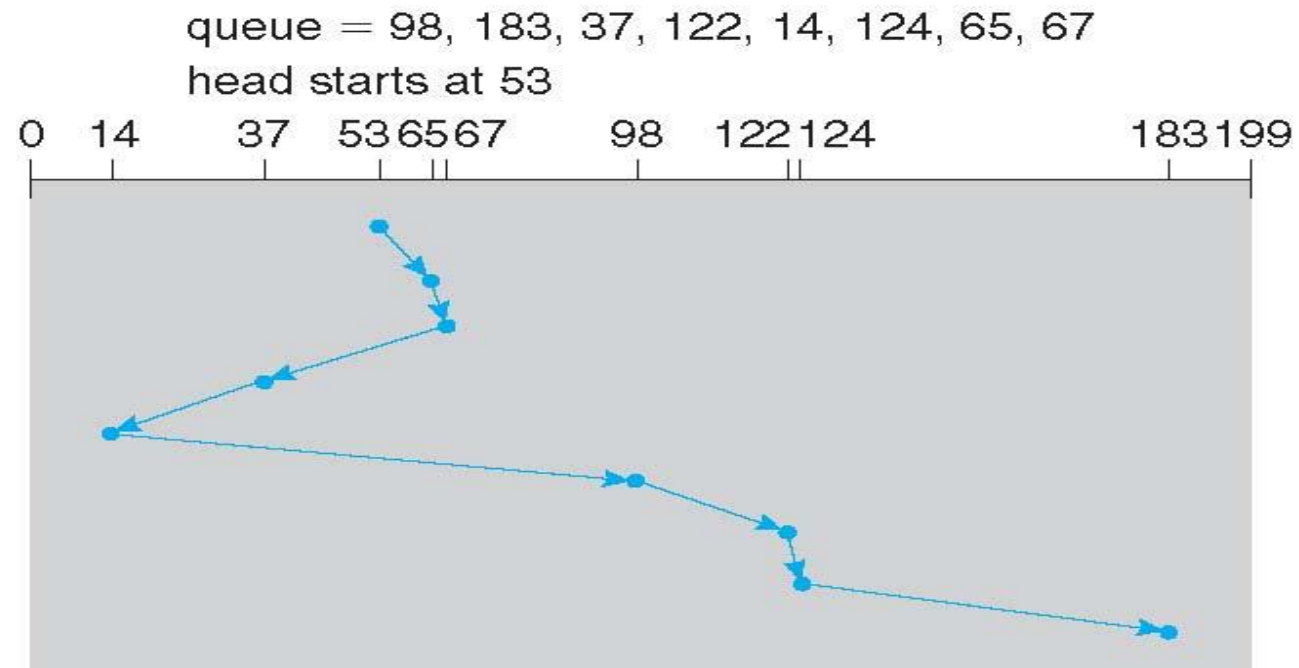
- Example -- total head movement of 640 cylinders



- Is it a good algorithm for elevator?

Shortest Seek Time First (SSTF)

- Selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests
- Example -- total head movement of 236 cylinders

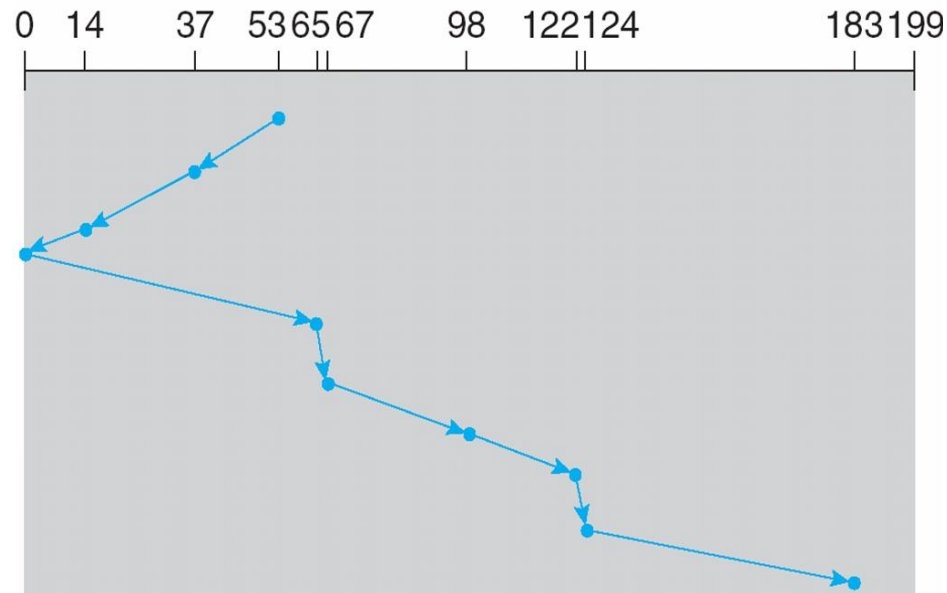


SCAN Algorithm

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed, and servicing continues.
- Illustration shows total head movement of 236 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



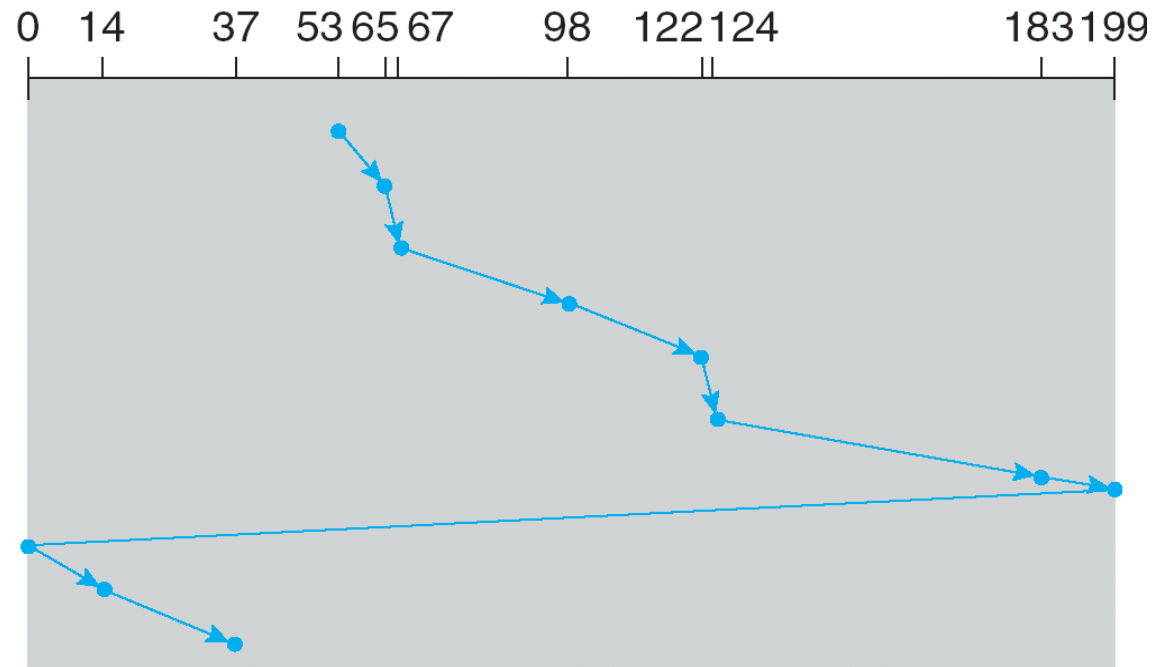
Circular Scan (C-SCAN)

- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
 - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Is it a good algorithm for elevator?

C-SCAN Example

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - Less starvation, but still possible

Error Detection and Correction

- Fundamental aspect of many parts of computing (memory, networking, storage)
- **Error detection** determines if there a problem has occurred (for example a bit flipping)
 - If detected, can halt the operation
 - Detection frequently done via parity bit
- Parity is one form of **checksum** – uses modular arithmetic to compute, store, compare values of fixed-length words
- Another error-detection method common in networking is **cyclic redundancy check (CRC)** which uses hash function to detect multiple-bit errors
- **Error-correction code (ECC)** not only detects, but can correct some errors
 - Soft errors correctable, hard errors detected but not corrected

Storage Attachment

- Computers access storage in three ways
 - Host-attached
 - Network-attached
 - Cloud
- Host attached access through local I/O ports, using one of several technologies
 - To attach many devices, use storage busses such as USB, firewire, thunderbolt
 - High-end systems use **fibre channel (FC)**
 - High-speed serial architecture using fibre or copper cables
 - Multiple hosts and storage devices can connect to the FC fabric

Example

- A disk system has 100 cylinders, numbered 0 to 99. Assume that the read/write head is at cylinder 50 and determine the order of head movement for FCFS, SSTF, SCAN and C-SCAN to satisfy the following stream of request. Also calculate total head travel distance and conclude which is better algorithm. To move from one cylinder to another cylinder will take 0.6 ms, so how much time each algorithm will take to complete all requests.
- stream of request : 30, 12, 62, 27, 67, 32, 83, 75, 85, 65, 17

Example (Ans)

Algorithm

Total Head Movement

FCFS

335

SSTF

108

SCAN

135

C-SCAN

180

RAID

Redundant Arrays of Independent Disks

Data Redundancy

- Data redundancy, although taking up extra space, adds to disk reliability.
- In case of disk failure, if the same data is also backed up onto another disk, we can retrieve the data and go on with the operation.
- If the data is spread across just multiple disks without the RAID technique, the loss of a single disk can affect the entire data.

Different RAID levels

- RAID-0 (Striping)
- RAID-1 (Mirroring)
- RAID 2 (Bit-Level Striping with Dedicated Hamming-Code Parity)
- RAID 3 (Bit-Level Striping with Dedicated Parity)
- RAID-4 (Block-Level Striping with Dedicated Parity)
- RAID-5 (Block-Level Striping with Distributed Parity)
- RAID-6 (Block-Level Striping with Distributed Double Parity)

RAID-0 (Stripping)

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

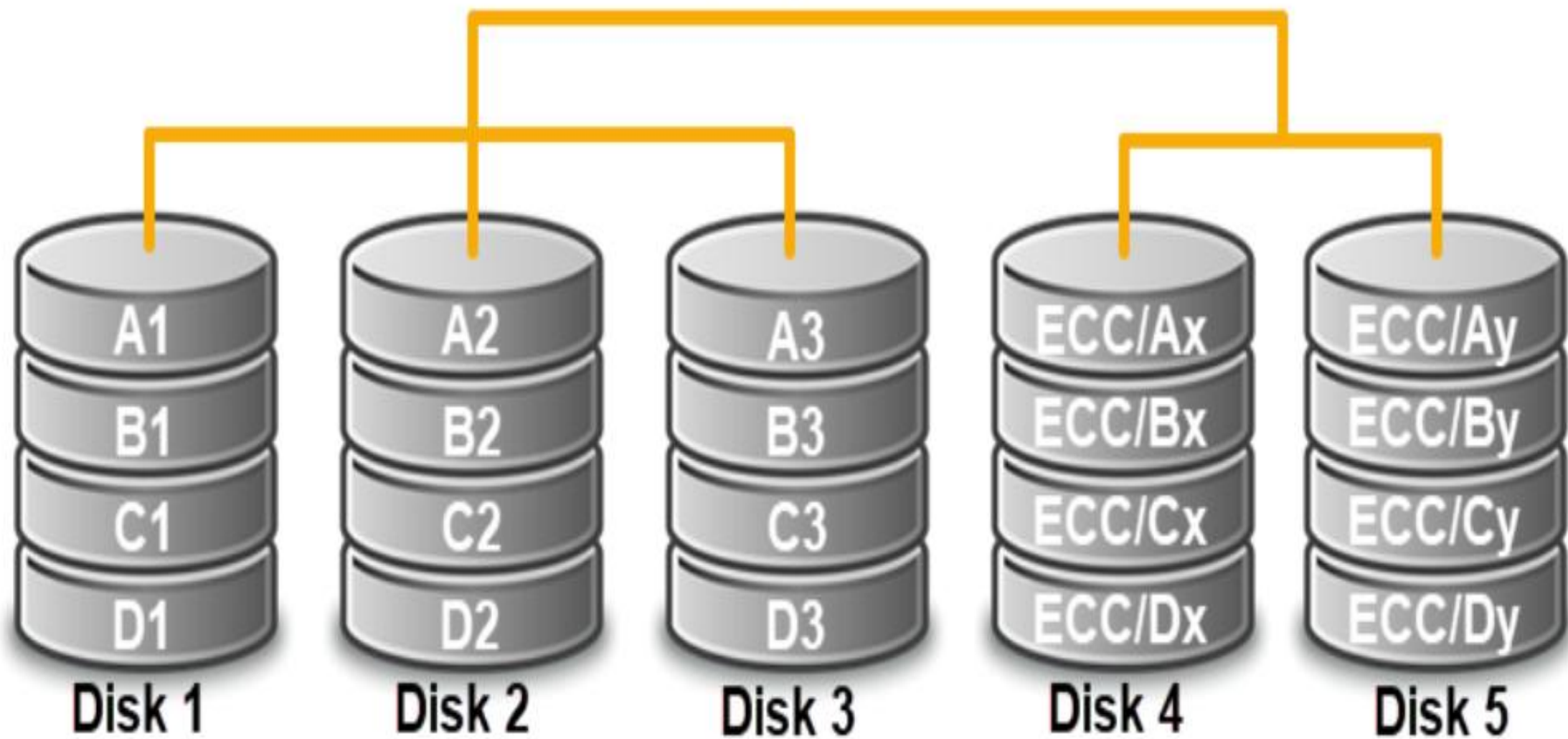
- Blocks are “stripped” across disks.
- In the figure, blocks “0,1,2,3” form a stripe.
- Instead of placing just one block into a disk at a time, we can work with two (or more) blocks placed into a disk before moving on to the next one.

RAID-1 (Mirroring)

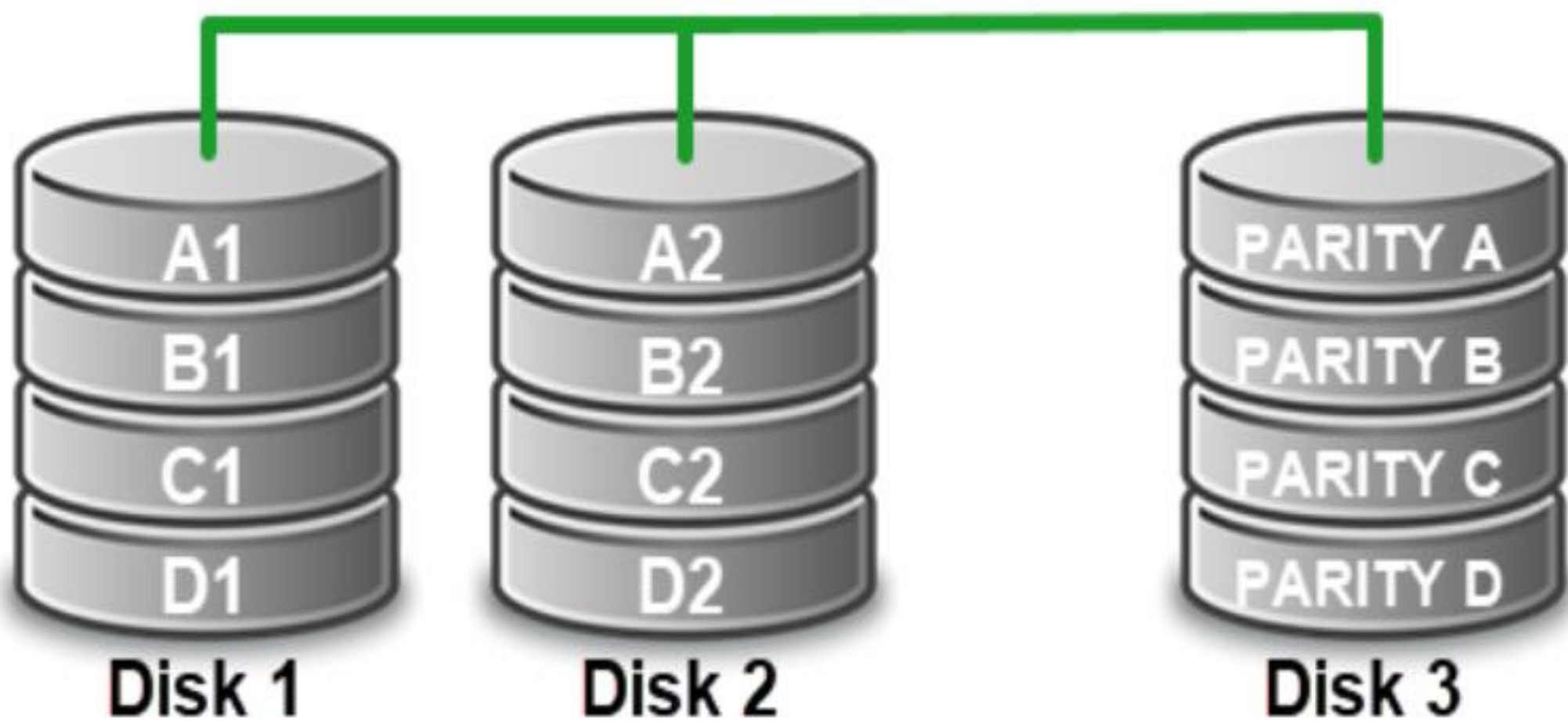
Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

- More than one copy of each block is stored in a separate disk. Thus, every block has two (or more) copies, lying on different disks.
- The figure shows a RAID-1 system with mirroring level 2.
- RAID 0 was unable to tolerate any disk failure. But RAID 1 is capable of reliability.

RAID 2



RAID 3



RAID-4 (Block-Level Striping with Dedicated Parity)

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

- Instead of duplicating data, this adopts a parity-based approach.
- In the figure, we can observe one column (disk) dedicated to parity.
- Parity is calculated using a simple XOR function. If the data bits are 0,0,0,1 the parity bit is $\text{XOR}(0,0,0,1) = 1$. If the data bits are 0,1,1,0 the parity bit is $\text{XOR}(0,1,1,0) = 0$. A simple approach is that even number of ones results in parity 0, and an odd number of ones results in parity 1.

C1	C2	C3	C4	Parity
0	0	0	1	1
0	1	1	0	0

RAID-5 (Block-Level Stripping with Distributed Parity)

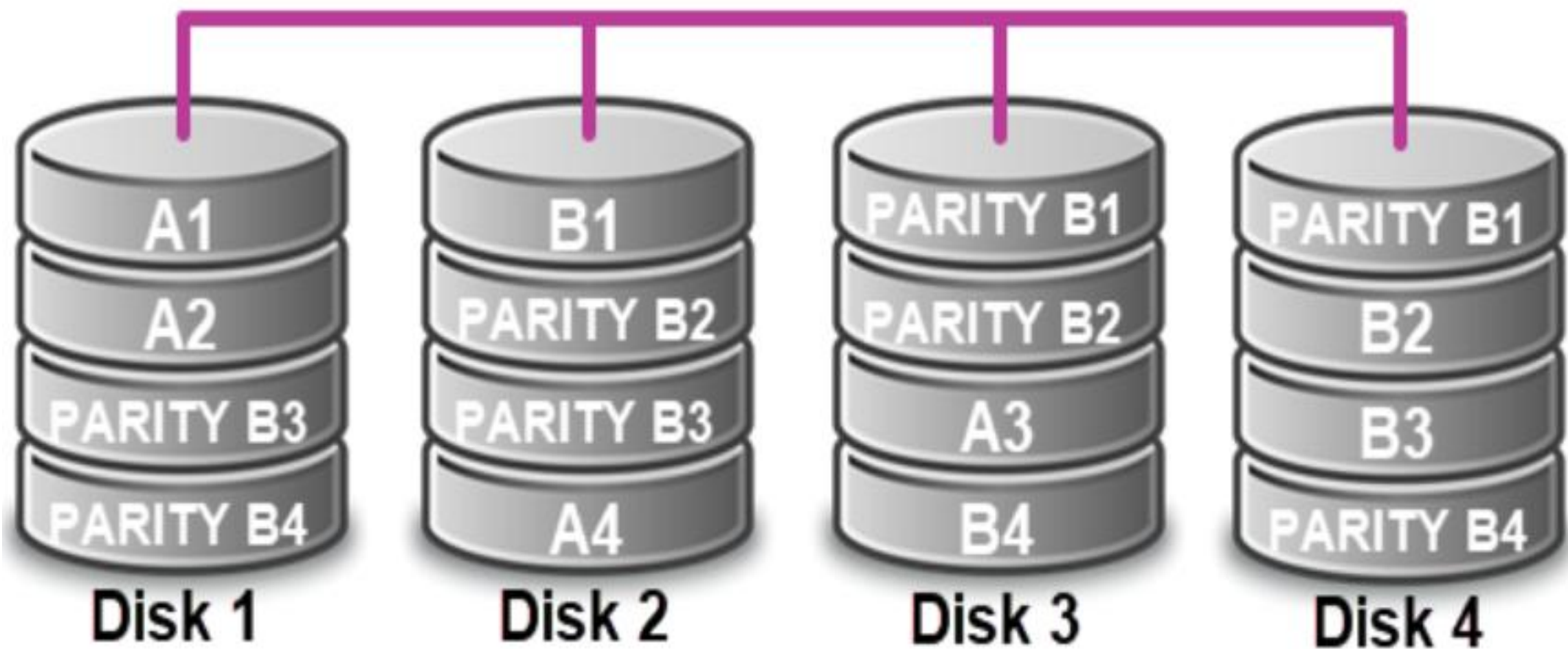
Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

- This is a slight modification of the RAID-4 system where the only difference is that the parity rotates among the drives.
- In the figure, we can notice how the parity bit “rotates”.
- This was introduced to make the random write performance better.

RAID-6 (Block-Level Striping with Distributed Double Parity)

- RAID 6 is same concept of RAID 5, but it have an extra parity bit storage than RAID 5.
- RAID 6 required minimum 4 disk.
- RAID 6 perform Byte-level striping with double distributed parity.
- In RAID 6, backup of parity storage will get store in another storage too. That means there are 2 parity backup

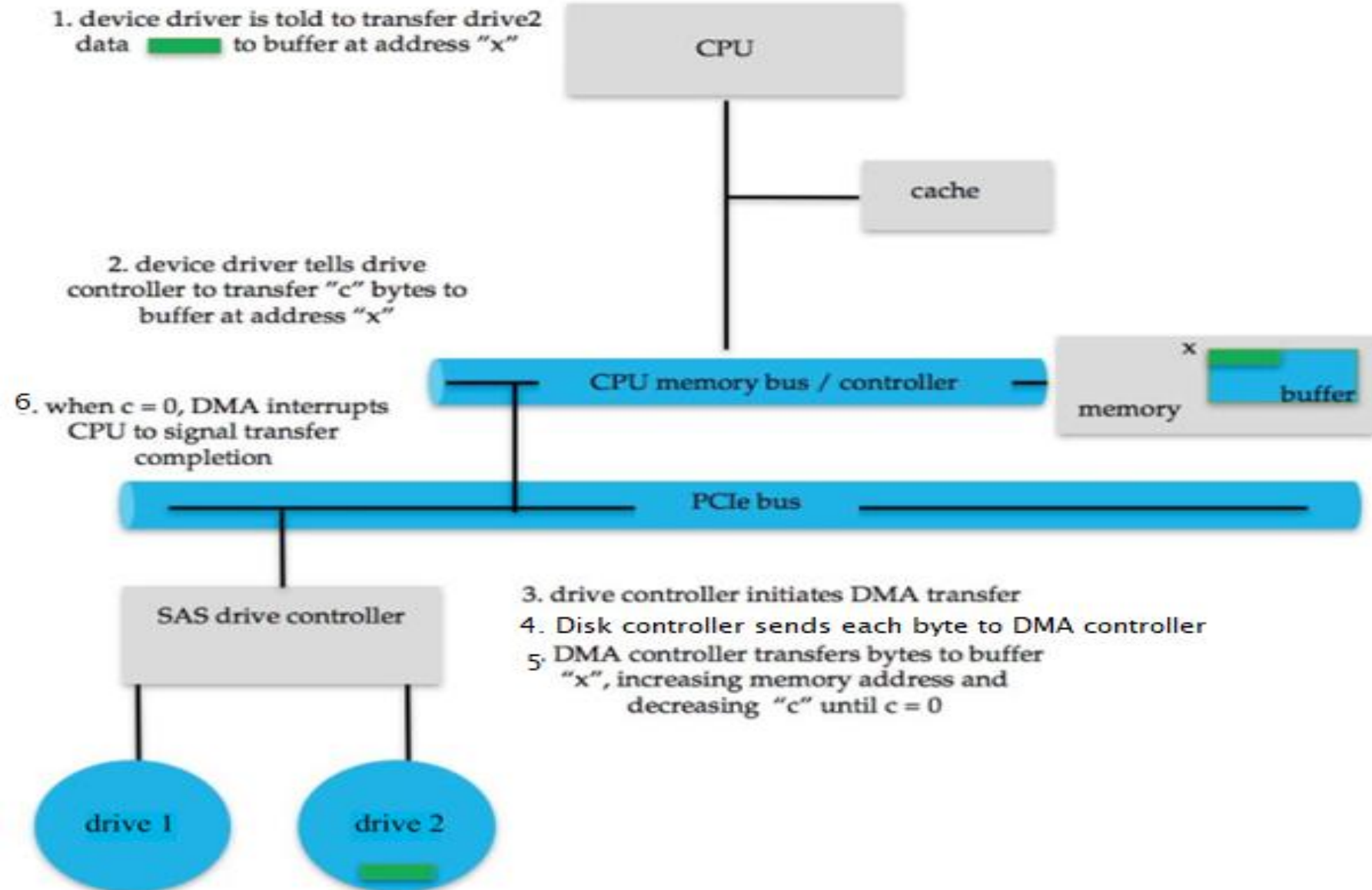
RAID 6



Direct Memory Access

- Used to avoid **programmed I/O** (one byte at a time) for large data movement
- Requires **DMA** controller
- Bypasses CPU to transfer data directly between I/O device and memory
- OS writes DMA command block into memory
 - Source and destination addresses
 - Read or write mode
 - Count of bytes
 - Writes location of command block to DMA controller
 - Bus mastering of DMA controller – grabs bus from CPU
 - **Cycle stealing** from CPU but still much more efficient
 - When done, interrupts to signal completion
- Version that is aware of virtual addresses can be even more efficient - **DVMA**

Six Step Process to Perform DMA Transfer



Improving Performance

- Reduce number of context switches
- Reduce data copying
- Reduce interrupts by using large transfers, smart controllers, polling
- Use DMA
- Use smarter hardware devices
- Balance CPU, memory, bus, and I/O performance for highest throughput
- Move user-mode processes / daemons to kernel threads